ASAP팀 기획제안서

팀 명	06. ASAP		
프로젝트 명	게임 AI개발을 통한 알고리즘 대결 플랫폼		
담당 교수	윤대균 교수님		
제출일	2026년 4월 10일		
팀원	학과	학번	이름
	소프트웨어학과	202220714	안주성
	소프트웨어학과	202126827	박서윤
	소프트웨어학과	202126860	이재혁

ASAP팀 기획제안서 1
1. 프로젝트 개요 3
1.1. 문제정의 및 솔루션 3
1.1.1. 문제정의 3
1.1.2. 솔루션 3
1.2. 주 타겟 및 Stakeholder 3
1.2.1. Target 3
1.2.2. Stakeholder 3
1.3. 핵심 가치 3
1.3.1. 참가자 3
1.3.2. 개최자 3
1.3.3. 기업 3
2. 기존 사례 분석 및 사업화 가능성 4
2.1. 유사 제품 / 서비스 4
2.1.1. 백준 온라인 저지, Codeforces, AtCoder 4
2.1.2. ALPHANO 4
2.1.3. 구름 IDE, 프로그래머스 4
2.2. 사업화 가능성 4
2.2.1. 비즈니스 모델 정의 4
2.2.2. 시장 규모 분석 5

2.2.3. 상용화 시 예상 사업 규모 5
3. User Story Mapping 6
3.1. 페르소나 정의 6
3.1.1. 참가자 6
3.1.2. 개최자 6
3.2. 사용자 여정 7
3.2.1. 참가자 7
3.2.2. 개최자 7
3.3. Epics & User Stories 8
Epic 1 : 대회 개최(Journey 3.1의 단계 4~8) 8
Epic 2 : 대회 중 중간 결과 발표(Journey 3.1의 단계 8) 8
Epic 3 : 대회 종료 및 결과 발표(Journey 3.1의 단계 9, 3.2의 단계 8) 8
Epic 4 : 대회 참가(Journey 3.2의 단계 2~4) 8
Epic 5 : 대회 진행(Journey 3.2의 단계 5~6) 9
4. 설계 및 구현 9
4.1. 기술 스택 9
4.1.1. 프론트엔드 9
4.1.2. 백엔드 9
4.1.3. 데이터베이스 11
4.1.4. 클라우드 11
4.1.5. 협업 도구 12
4.2. 시스템 설계 13
4.2.1. 시스템 개요 13
4.2.2. 소프트웨어 아키텍처 15
프론트엔드 (Presentation Layer) 15
백엔드 (Application Layer) 16
데이터 계층 16
4.2.3. 데이터 정의 17
4.2.4. API 정의 21
5. 과제 수행 계획 23
5.1. 업무분장 23
5.1.1. 개발 업무 분장 23
5.1.2. 비 개발 분야 업무분장 24
5.2. 협업 및 데브옵스 자동화 25
5.2.1. 커밋 컨벤션 25
5.2.2. 브랜치 전략 25
5.3. 성과 측정 방안 26
5.3.1. 정량적 KPI 26
5.3.2. 정성적 KPI 27

1. 프로젝트 개요

1.1. 문제정의 및 솔루션

1.1.1. 문제정의

- 생성형 AI의 발전으로 인해 기존 온라인 알고리즘 대회(Codeforces, AtCoder)에 부정행위가 많아지고 있다.
- 일반 유저들은 알고리즘 실력을 키우며 자신의 순위가 오르는 것을 보며 성취감을 얻지만, AI 부정사용자들의 증가로 자신의 순위가 떨어져 대회 참여 동기가 줄어드는 상황이다.
- 기존 알고리즘 대회는 근본적으로 AI를 막을 수 없는 구조이다.

1.1.2. 솔루션

- AI를 사용해도 문제가 되지 않는 알고리즘 대회를 개최하는 것으로 기존 문제 해결할 수 있다.
- 일반 유저가 이 방식의 알고리즘 대회를 개최 가능한 플랫폼을 개발하여 문제를 해결하고자 한다.

1.2. 주 타겟 및 Stakeholder

1.2.1. Target

- 자신의 알고리즘 실력과 더불어 게임 전략을 남들과 경쟁하고 싶은사람
- 기존 알고리즘 대회에서 생성형 AI에 의해 흥미가 떨어진 사람

1.2.2. Stakeholder

- 대회를 개최하고 싶어하는 개인, 기관, 단체
- 문제 해결 능력을 갖춘 SW 인재를 확보하고 싶어하는 기업
- 자신의 게임을 홍보하고 싶은 게임개발자

1.3. 핵심 가치

1.3.1. 참가자

- 공정성이 확보된 환경에서 합리적 경쟁이 가능하다.
- 알고리즘 구현 실력을 증진할 수 있다.
- 문제 해결 능력을 향상시킬 수 있다.

1.3.2. 개최자

- 본인이 설계한 게임 규칙으로 대회를 개최할 수 있다.
- 자신이 만든 게임 대회 주제로 개최하여 홍보할 수 있다.

1.3.3. 기업

- 문제 해결 능력을 갖춘 SW 인재 확보에 유리하다.
- 개인의 사고력과 개발 능력을 평가할 수 있다.

2. 기존 사례 분석 및 사업화 가능성

2.1. 유사 제품 / 서비스

2.1.1. 백준 온라인 저지, Codeforces, AtCoder

- 개인 유저가 알고리즘 대회를 개최할 수 있다.
- 알고리즘 대회가 주기적으로 열리며, 참가자는 대회 결과에 따른 레이팅이 반영된다.

2.1.2. ALPHANO

- 강화학습 온라인 저지 서비스를 제공한다.
- 운영자만 대회를 개최할 수 있다.
- 비슷한 레이팅 참가자 간 매치메이킹 시스템만을 제공한다.

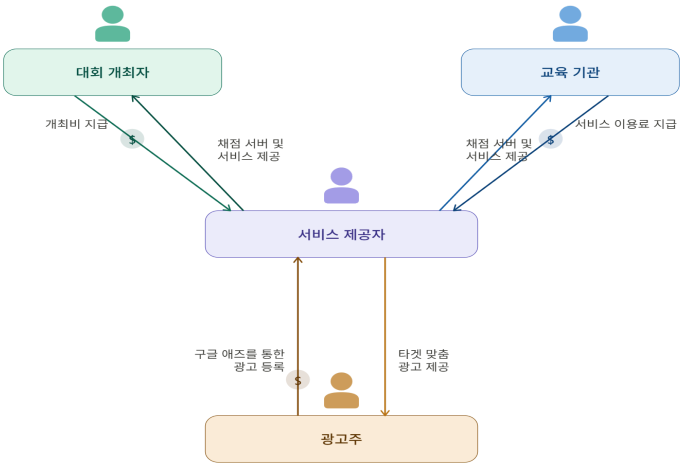
2.1.3. 구름 IDE, 프로그래머스

- 알고리즘 테스트와 과제형 평가를 통해 실무 역량을 객관적으로 측정할 수 있는 기능을 제공해준다.
- 기관 및 단체에게 문제 채점 서비스를 제공해준다.

2.2. 사업화 가능성

2.2.1. 비즈니스 모델 정의

- 서비스 제공자는 대회 개최자 및 교육기관에게 채점 서버 및 관련 서비스를 제공한다.
- 대회 개최자는 서비스 제공자에게 개최비를 지급한다.
- 교육기관은 서비스 이용료를 서비스 제공자에게 지급한다.
- 서비스 제공자는 광고주에게 타겟이 명확한 광고 홍보 효과를 제공한다.
- 광고주는 구글 애즈를 통해 플랫폼에 광고를 등록한다.



[그림1] 비즈니스 모델

2.2.2. 시장 규모 분석

- 본 서비스는 AI를 활용하더라도 최적해를 도출할 수 없는 구조이므로, 개인의 사고력과 전략이 핵심이 된다. 이런 특징이 학습적 측면에서 잠재 가치가 있다고 판단하여 시장 분석을 대학생 중심으로 진행했다.
- 서비스의 전체 시장(TAM)은 대한민국 전체 대학생 수인 약 300만 명으로 설정했다.
- 유효시장(SAM)은 IT 계열 학과 재학생과 복수전공자를 기준으로 약 13만 명으로 추정했다.
- 수익시장(SOM)은 구름 IDE와 B2B를 체결한 교육기관의 재학생 수를 기반으로 약 3만 명으로 산정했다.

2.2.3. 상용화 시 예상 사업 규모

- ARPU
  - 한 대회의 개최기간은 1주~2주, 인증대회 개최자들은 다른 대회가 열려있는 동안에는 개최를 하고싶지 않을 것으로 판단하여 한달 평균 2~5회 정도 열릴 것으로 예상하였다.
  - 인증대회의 개최비의 경우 사용자수, CPU, Memory 자원을 고려하여 100,000 + α 정도로 추산하여, 대회 개최비를 통한 연 매출을 2,500,000원으로 가정하였다.
  - 일 예상 사이트 방문자 수를 nypc 대회 참가자 수인 5000명 중 20%로 가정하여 1000명, 일 방문 횟수를 10회 정도로 고려하여 구글애즈를 통한 연 수익을 10,000,000원으로 가정하였다.
  - 교육 기관의 경우 구름 IDE를 사용하는 대학교 수의 10%를 고려하여 초기 타겟 시장을 선정했다. 서비스 사용자 1인당 15,000원으로 서비스 사용료를 측정하였다. 교육기관을 통한 연 매출은 540,000,000으로 추정하였다.

항목	매출(년)
대회 개최비	년 예상 인증 대회 개최 수 x 100,000원 + 년 예상 비인증 대회 개최 수 x 10,000원 = 2,500,000원
구글 애즈	일 예상 사이트 방문자 수 x 광고단가 x 365(일) = 10,000,000원

교육 기관	수익시장 인원수 x 초기 타겟 시장 가능성(0.1) x 1.5만원 = 540,000,000원

[표1] 예상 매출

- 초기 서비스 운영 기간 동안은 교육기관과의 B2B 체결이 이루어지기 힘들 것으로 예상한다. 대회 개최비와 구글 애즈를 통한 서버 유지비를 통해 서비스를 유지하는 구조로 설계하였다. 서비스의 인지도가 늘어남에 따라 많은 교육기관과의 B2B 체결을 핵심 수익으로 가정하였다.

3. User Story Mapping

3.1. 페르소나 정의

3.1.1. 참가자

- 이름 : 백 준호 (23세, 대학생, 프로그래밍에 관심이 많음)
  - 직업 : 대학생
  - 거주지 : 서울, 대한민국
  - 교육 수준 : 서울 소재 4년제 대학교 컴퓨터공학과 3학년
  - 소득 수준 : 월 70~90만 원
  - 주력 언어: C++
  - 자주 쓰는 플랫폼: 백준(Baekjoon), 프로그래머스, LeetCode, Codeforces
  - 성격 : 집요함과 승부욕
- 이름 : 박 영훈 (29세, 직장인)
  - 직업 : 외국계 IT 기업 백엔드 개발자 (4년 차)
  - 거주지 : 경기도 성남시 분당구 정자동
  - 교육 수준 : 수도권 4년제 대학교 컴퓨터공학과 졸업
  - 소득 수준 : 연 7,000만 원
  - 주력 언어: Python, Java
  - 유니콘 기업으로 이직을 준비 중이다.
  - 취미로 알고리즘 대회에 계속 참여 중이다.

3.1.2. 개최자

- 이름 : 염 승민 (32세, 직장인)
  - 직업 : 중견 IT 서비스 기업 인사팀 인재채용 파트 과장
  - 거주지 : 경기도 용인시 기흥구
  - 교육 수준 : 서울 소재 4년제 대학교 졸업(경영학과)

    - 소득 수준 : 연 4,500만 원
    - 경영학 전공자로서 채용 프로세스의 '비용 대비 효율(ROI)'을 중요하게 생각한다.
    - 최근 생성형 AI가 개발자 채용 시장에 미치는 영향에 대해 고민하고 있으며, 기존 코딩 테스트의 변별력 하락을 실감하고 있다.
    - 단순히 기술 스택이 화려한 지원자보다, 논리적인 전략을 세우고 이를 코드로 구현할 줄 아는 '진짜 실력자'를 찾는 데 열중한다.
- 이름 : 김 인용 (22세, 대학생)
    - 직업 : 인디게임 개발자
    - 거주지 : 경기도 수원시 권선구
    - 교육 수준 : 4년제 대학교 졸업(미디어학과)
    - 소득 수준 : 연 2,800만 원
    - 자신의 게임을 특이한 방식으로 홍보하고 싶어한다.
    - 인디게임 개발자로서 홍보 수단이 한정적이다.

3.2. 사용자 여정

3.2.1. 참가자

1. 사이트 접속
2. 로그인
3. 현재 개최중인 대회 확인
4. 게임 주제 확인 및 대회 참가
5. “배틀 코드” 개발 및 코드 제출
6. 중간 대전 결과 확인
7. 개선 후 반복
8. 대회 종료

3.2.2. 개최자

1. 개최할 게임 주제 구상
2. 사이트 접속
3. 로그인
4. 카테고리 선택(인증, 비인증)
5. 주제 명세서, 채점 코드, 샘플 AI코드, 시각화 리소스 제출
6. 기간 설정 후 대회 개최
7. 개최 예정 대회 내용 수정
8. 중간 결과 발표
9. 대회 종료 및 결과 발표

3.3. Epics & User Stories

Epic 1 : 대회 개최(Journey 3.1의 단계 4~8)

- User Story 1.1 : 나는 인사팀 채용담당자로서, 대회를 개최하고 싶다 -> SW인재를 확보하기 위해
- User Story 1.2 : 나는 개최자로서, 대회 명세, 채점 코드, 예제 배틀 코드, 시각화 리소스를 제출하고 싶다. → 참가자들의 "배틀 코드"를 공정하게 평가하고 대전 과정을 시각적으로 확인하기 위해
- User Story 1.3 : 나는 개최자로서, 대회 정보를 입력하고 싶어 -> 참가자들에게 개최할 대회의 정보를 제공하기 위해
- User Story 1.4 : 나는 개최자로서, 개최 예정인 대회를 수정하고 싶어 -> 문제가 생길 시 기한을 재설정 하거나 내용 및 오류를 수정하기 위해

Epic 2 : 대회 중 중간 결과 발표(Journey 3.1의 단계 8)

- User Story 2.1 : 나는 개최자로서, 주기적으로 중간 결과를 공지하고 싶다 -> 대결 결과를 통해 참가자들이 자신의 코드를 수정할 수 있도록 하기 위해

Epic 3 : 대회 종료 및 결과 발표(Journey 3.1의 단계 9, 3.2의 단계 8)

- User Story 3.1 : 나는 개최자로서, 대회를 잘 마무리 하고 싶다 -> 유저들에게 대회 결과 및 대회 기록 등을 제공하여 다음 대회 참여 욕구를 증진시키기 위해
- User Story 3.2 : 나는 참가자로서, 대회 결과를 확인하고 싶다 -> 등수를 확인하여 나의 객관적인 실력 확인 및 성취감을 얻기 위해
- User Story 3.3 : 나는 인사팀 채용담당자로서, 대회 결과에 따른 보상을 주고 싶다 -> SW인재를 확보하여 기업 경쟁력을 강화시키기 위해
- User Story 3.4 : 나는 참가자로서, 최종 대회는 풀리그로 진행됐으면 좋겠어 -> 토너먼트나 스위스 방식으로 진행될 경우, 조 편성에 따른 운적 요소가 존재하기 때문에 공정했으면 좋겠어

Epic 4 : 대회 참가(Journey 3.2의 단계 2~4)

- User Story 4.1 : 나는 참가자로서, 로그인을 하고싶어 -> 내가 제출한 코드들을 저장하고, 대회에 참가하기 위해
- User Story 4.2 : 나는 참가자로서, 현재 개최 중인 대회를 확인하고 싶어 -> "코드 배틀" 대회를 참가해 나의 알고리즘 실력과 게임 전략을 남들과 겨뤄보고 싶기 때문에

Epic 5 : 대회 진행(Journey 3.2의 단계 5~6)

- User Story 5.1 : 나는 참가자로서, 내가 개발한 배틀 코드를 작성하고 제출하고 싶어 -> 내가 만든 “배틀 코드”를 다른 참가자들과 겨뤄보기 위해
- User Story 5.2 : 나는 참가자로서, 주기적으로 다른 사람과의 대결 결과를 확인하고 싶다 -> 대결 결과를 통해 나의 “배틀 코드”를 발전시키기 위해

4. 설계 및 구현

4.1. 기술 스택

4.1.1. 프론트엔드

TypeScript	대회 목록, 제출 결과, 리더보드 등 다양한 데이터 구조를 다루는 본 프로젝트에서 타입을 명확히 정의하면 런타임 오류를 사전에 방지하고, 프론트엔드-백엔드 간 API 응답 타입 불일치 문제를 컴파일 단계에서 잡아낼 수 있어 채택하였다.
React	대회 목록, 코드 에디터, 리더보드, 대전 시각화 등 동적이고 복잡한 UI를 컴포넌트 단위로 분리하여 효율적으로 관리할 수 있다. DOM(Document Object Model)은 웹 페이지의 구조를 객체 형태로 표현한 것으로 가상 DOM을 통해 리더보드 실시간 갱신과 같은 빈번한 UI 업데이트를 성능 저하 없이 처리할 수 있으며, 팀원이 사용 경험이 있어 학습 비용 없이 바로 개발에 착수할 수 있기에 채택하였다.
Zustand	React의 Context API는 상태를 전역적으로 공유할 수 있으나, 컴포넌트 수가 증가할수록 불필요한 리렌더링이 발생하고 상태 관리가 복잡해질 수 있다. 이에 따라 Zustand를 도입하여 전역 상태를 효율적으로 관리하였다. Zustand는 상태 정의와 사용에 필요한 반복적인 코드(보일러플레이트)가 적어 간단한 구조로 상태를 관리할 수 있으며, 로그인 사용자 정보, 현재 참가 중인 대회 상태, 코드 에디터 내용 등을 컴포넌트 트리 외부에서 독립적으로 관리할 수 있다. 이러한 특성을 바탕으로 상태 관리의 복잡도를 줄이고 성능을 개선하기 위해 해당 기술을 채택하였다.

[표 2] 프론트엔드 기술 스택

4.1.2. 백엔드

Java	Java는 백엔드 개발에 널리 사용되는 객체지향 프로그래밍 언어이다. 높은 점유율 을 기반으로 많은 프레임워크들이 지원되며, 본 프로젝트에서 백엔드 서버로 사용할 Spring Boot의 경우 Java를 사용하므로 채택하였다.
Spring Boot	본 프로젝트에 Spring Boot를 채택한 이유는 기한 내 완성을 위한 높은 개발 생산성 때문이다. 특히 Redis와의 연동성이 뛰어나, Redis를 메시지 브로커로 활용해 C++ 채점 서버와 분리하고 안정적인 비동기 통신을

	구현하기 용이하다. 또한 내장 웹 서버와 JPA를 통해 Docker, AWS, PostgreSQL 기반의 클라우드 환경 배포에 매우 적합하고 팀원들 중 일부가 사용 경험이 있어 채택하였다.
Spring Data JPA	Spring Data JPA는 객체와 데이터베이스 간 매핑을 자동으로 처리하는 ORM(Object Relational Mapping)기술이다. SQL 작성 없이 객체 중심의 데이터 접근이 가능하다. 데이터 처리 로직을 단순화하고 유지보수성을 향상시키기에 채택하였다.
QueryDSL	복잡한 조회 조건이나 동적 쿼리가 필요한 경우를 위해 QueruDSL을 채택하였습니다. 예를 들어, 특정 조건에 따른 결과 필터링이나 검색 기능 구현 시 타입 안전한 방식으로 쿼리를 작성하여 안정성과 가독성을 높일 수 있채택하였다.
C++	채점 서버는 C++로 구현하였다. AWS EC2환경에서 구동되며 채점 서버의 특성상 빠른 로직 처리가 중요하기에 채택하게 되었다.
Swagger	Swagger는 REST API를 기반으로 명세를 자동 생성하고 이를 웹 UI를 통해 확인할 수 있도록 지원하는 도구이다. 이를 통해 프론트엔드와 명세 기반의 협업을 원활하게 수행할 수 있으므로 해당 기술을 채택하였다.
JUnit	자바와 Spring Boot를 사용하는 백엔드 환경에서는 소스 코드와 테스트 코드가 `main`과 `test` 디렉토리로 분리되어 관리된다. JUnit 5는 자바에서 널리 사용되는 테스팅 프레임워크로, 코드 구조에 맞춰 테스트 클래스를 분리하여 Controller, Service, Repository 등 각 레이어에 대한 단위 테스트를 수행할 수 있도록 지원한다. 이를 통해 기능별 및 모듈별 정상 동작을 검증하여 오류를 조기에 발견하고 개발 일정을 안정적으로 관리할 수 있으므로 해당 기술을 채택하였다.
JaCoCo	JaCoCo는 테스트 실행 결과를 기반으로 코드 커버리지를 측정하는 도구로, 작성된 테스트 코드가 실제 자바 코드의 어느 범위까지 검증하고 있는지를 시각적으로 확인할 수 있도록 지원한다. 이를 통해 테스트 품질을 일정 수준 이상으로 유지하고, 테스트의 누락 영역을 식별하여 보완할 수 있으므로 해당 기술을 채택하였다.
Insomnia	Insomnia는 REST API의 테스트 및 디버깅을 위한 개발 도구로, 본 프로젝트에서 작성한 API의 요청 및 응답을 검증하기 위해 채택하였다. 또한 API를 CRUD 메서드 단위로 직접 호출하고 검증함으로써, API의 정상 동작을 사전에 확인할 수 있도록 구성하였다.

[표 3] 백엔드 기술 스택

4.1.3. 데이터베이스

PostgreSQL	MySQL 대비 SQL 표준 준수율이 높고, JSONB 타입을 지원하여 대전 턴별 로그와 같은 반구조화 데이터를 별도 NoSQL 없이 유연하게 저장할 수 있다. 리더보드 순위 계산같은 복잡한 집계 쿼리에서도 우수한 성능을 제공하여 채택하였다.
Redis	Redis는 메모리 기반 key-value 저장소이며 매우 빠른 데이터 처리 속도를 제공한다. 향후 채점 서버 구현 시 Spring Boot와 C++ 서버 간 비동기 메시지 큐로 사용하기 위해 채택하였다.

[표 4] 데이터베이스 기술스택

4.1.4. 클라우드

AWS EC2	AWS에서 제공하는 EC2는 가상 서버 공간을 탄력적으로 제공하는 클라우드 서비스이다. Spring Boot 서버와 C++ 채점 서버를 구동하는 가상 서버 환경으로 적절하다고 판단하였고, 사용량 비례 과금으로 초기 비용 부담이 적고 대회 기간 중 트래픽 증가 시 인스턴스를 탄력적으로 확장할 수 있어 채택하였다.
AWS S3	React 빌드 결과물을 호스팅하고, 참가자가 제출한 배틀 코드와 개최자가 업로드한 대회 자료(명세서, 채점 코드, 시각화 리소스)를 저장하는 객체 스토리지이다. 프론트엔드 정적 호스팅과 파일 저장소를 하나의 서비스로 통합할 수 있어 채택하였다.
AWS RDS	Amazon Web Services의 Amazon RDS는 관계형 데이터베이스를 클라우드 환경에서 손쉽게 구축하고 운영할 수 있도록 지원하는 관리형 서비스이다. 본 시스템에서는 PostgreSQL을 RDS 기반으로 운영하여 자동 백업, 장애 복구, 패치 관리 등의 기능을 활용함으로써 데이터베이스 운영 부담을 최소화한다. 또한 EC2에 직접 데이터베이스를 설치하여 운영하는 방식에 비해 높은 안정성과 운영 편의성을 제공하므로 해당 기술을 채택하였다.
CloudFront	CloudFront의 엣지 로케이션 캐싱을 통해 사용자와 가까운 서버에서 프론트엔드 정적 파일을 전달하여 응답 속도를 높이고, HTTPS를 기본 제공하여 보안성도 확보하기 위해 채택하였다.
Docker	Docker는 애플리케이션을 컨테이너 단위로 실행할 수 있는 개발 도구이다. 참가자가 제출한 배틀 코드를 격리된 컨테이너 환경에서 실행하여 보안과 공정성을 보장한다. 모든 참가자에게 동일한 컴퓨팅 자원(CPU, 메모리, 시간 제한)을 제공하며, 악성 코드가 호스트 시스템에 영향을 미치지 못하도록 격리하는 것이 알고리즘 대결 플랫폼의 핵심 요구사항이므로 채택하게 되었다.

Nginx	Nginx는 리버스 프록시 서버로서 클라이언트의 요청을 EC2 내 Spring Boot 서버로 라우팅하는 역할을 수행한다. 이를 통해 엔드포인트를 단일화하고 로드 밸런싱을 적용하여 트래픽을 효율적으로 분산할 수 있으며, Blue-Green 방식의 무중단 배포를 지원하여 서비스 운영 중에도 안정적인 배포가 가능하다. 시스템의 안정성과 운영 효율성을 향상시키기 위해 해당 기술을 채택하였다.
Github Actions	GitHub Actions는 GitHub에서 제공하는 CI/CD 자동화 도구로, 코드 변경 사항을 감지하여 빌드와 테스트, 배포 과정을 자동으로 수행한다. 프론트엔드와 백엔드에서 발생한 코드 변경은 GitHub Actions를 통해 감지되며, GitHub에서 제공하는 가상 환경에서 각각 빌드 및 테스트를 수행한다. 이후 정상 동작이 확인된 경우 클라우드 환경의 EC2와 S3에 자동으로 배포된다. 또한 main 브랜치와 그 외 브랜치를 구분하여 각각 독립된 환경에서 테스트 및 배포가 이루어지도록 구성함으로써, 기능별 오류를 사전에 검출하고 안정적인 배포 환경을 유지할 수 있다. 개발 효율성과 코드 품질을 향상시킬 수 있으므로 해당 기술을 채택하였다.

[표 5] 클라우드 기술 스택

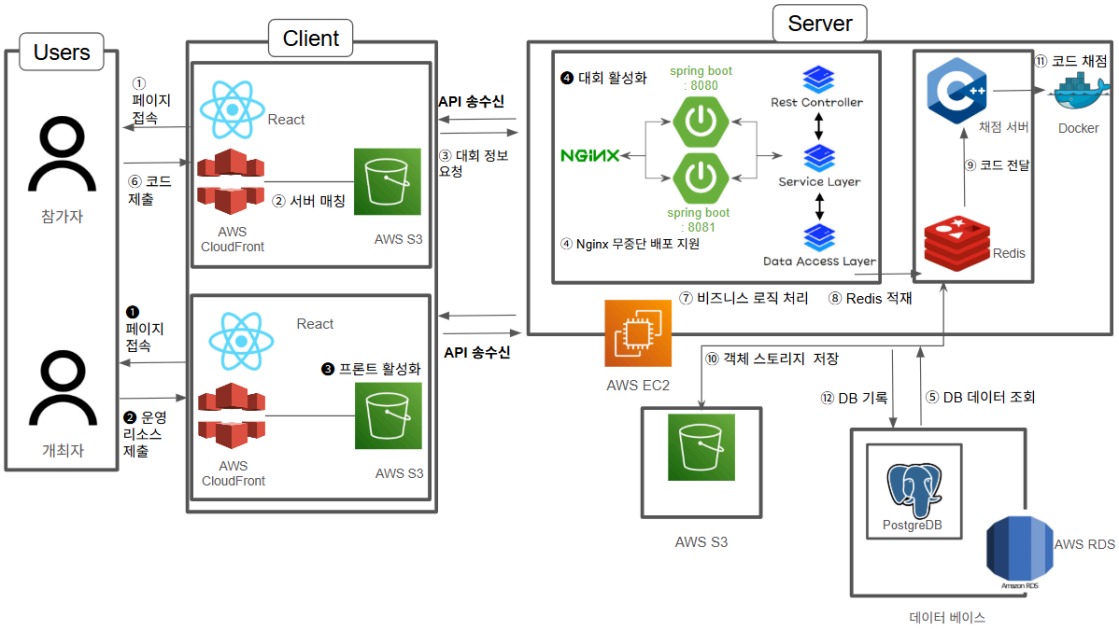
4.1.5. 협업 도구

Slack	팀원간 소통 및 협업 도구로 슬랙을 채택하였다. 개발 관련된 정보 및 자료를 채널을 통해 공유하고 공지사항을 관리한다.
Git/Github	소스 코드 버전 관리와 브랜치 전략(main/server/dev)을 통해 안정적인 협업 흐름을 유지한다. 하나의 레포지토리에서 모노레포 형식으로 프론트와 벡엔드를 관리한다. main 브랜치는 실제 배포되는 브랜치이며, server, client브랜치를 나누어 충돌을 최소화하며 사용할 계획이다.
Google Docs	기획서, 회의록, API 명세 등 문서를 실시간 공동 편집하여 팀원 간 정보 동기화를 빠르게 할 수 있어 채택했다.
Burndown Chart	개발 목록을 프로덕트 백로그로 관리할 시 해당 백로그를 기반으로 스프린트를 생성하고 역할을 분담하며 개발 일정을 체계적으로 관리하는 도구로 Burndown Chart를 채택하였다.
Github Gemini Code Review	PR 생성 시 Gemini AI가 자동으로 코드를 리뷰하여 버그와 보안 취약점을 1차로 감지한다. 또한 개선 가능하거나 오류인 코드에 대해서 리팩토링 기능을 제공하여 사용자의 간단한 확인만으로 코드를 수정하는 기능을 제공다. 3인으로 구성된 소규모 팀에서 코드 리뷰 부담을 줄이면서도 코드 품질을 유지할 수 있으므로 채택하였다.
Discord	팀원들이 모두 사용 경험이 있는 화상 회의 도구이므로, 비대면 회의 때 사용하기 위해 채택하였다.

[표 6] 협업 도구

4.2. 시스템 설계

4.2.1. 시스템 개요



[그림 2] 시스템 전체 개요

위 그림은 전체 시스템 개요를 도식화한 모식도이다. 크게 대회에 참가하는 참가자, 대회를 개최하는 개최자를 User영역, User가 서비스에 접근하는 기능을 포함한 Client영역, User가 Client를 통해 보낸 정보를 처리하는 Server영역으로 나누어진다. 전체적인 과정 설명은 다음과 같다.

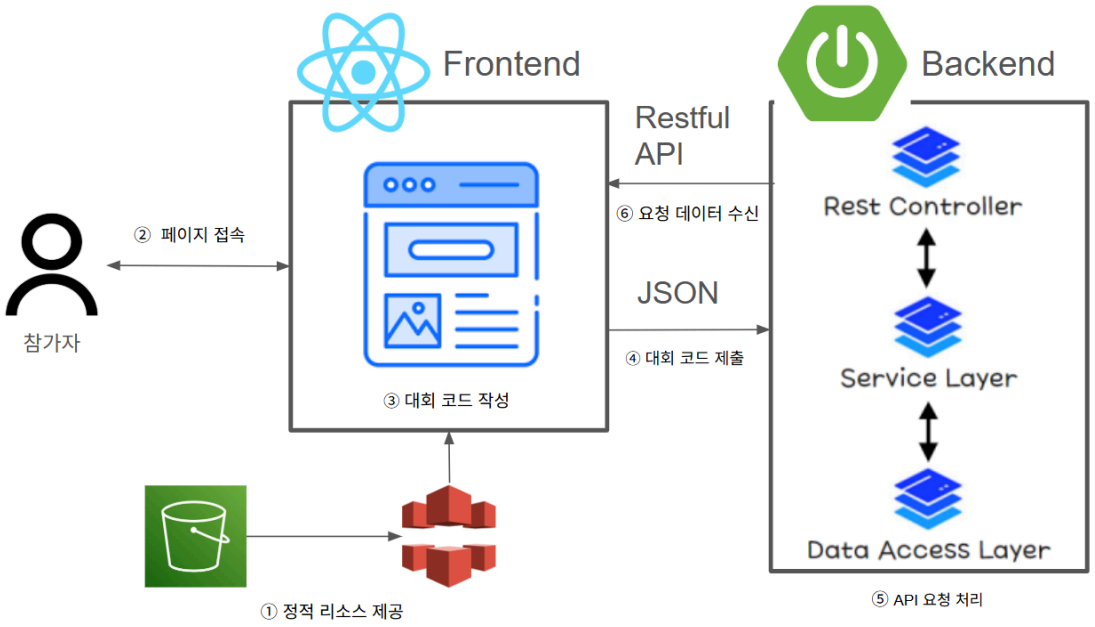
- ① 페이지 접속: 참가자가 AWS CloudFront를 통해 React 기반의 웹 페이지에 빠르고 안전하게 접속한다.
- ② 서버 매칭: CloudFront가 S3에 호스팅된 프론트엔드 정적 리소스를 참가자의 브라우저로 전달하여 화면을 렌더링한다.
- ③ 대회 정보 요청: 참가자가 대회 목록이나 상세 내용을 보기 위해 백엔드 서버(API)로 데이터를 요청한다.
- ④ **Nginx** 무중단 배포 지원: Nginx의 라우팅 기능을 활용하여 서비스 중단 없이(Blue-Green) 백엔드 서버를 안정적으로 배포 및 운영한다.
- ⑤ **DB** 데이터 조회: Spring Boot 서버가 AWS RDS(PostgreSQL)에 접근하여 대회 정보 및 리더보드 데이터를 가져온다.
- ⑥ 코드 제출: 참가자가 웹 에디터를 통해 작성한 배틀 코드를 백엔드 서버로 전송한다.

- ⑦ 비즈니스 로직 처리**:** 백엔드 서버가 제출된 코드의 유효성을 검사하고 데이터 처리를 수행한다.
- ⑧ **Redis** 적재**:** 서버의 부하를 막기 위해 제출된 코드의 채점 요청을 Redis 메시지 큐에 비동기 작업으로 등록한다.
- ⑨ 코드 전달**:** 대기 중인 C++ 채점 서버가 Redis 큐에서 채점할 작업(배틀 코드)을 꺼내온다.
- ⑩ 객체 스토리지 저장**:** 참가자가 제출한 코드 원본 파일은 백업 및 향후 조회를 위해 AWS S3 객체 스토리지에 영구 저장한다.
- ⑪ 코드 채점**:** C++ 채점 서버가 Docker의 격리된 컨테이너 환경 내에서 배틀 코드를 실행하여 공정하게 대전을 시뮬레이션한다.
- ⑫ **DB** 기록**:** 채점 및 대전 시뮬레이션이 완료된 최종 결과를 RDS(PostgreSQL)에 업데이트하여 순위에 반영한다.

- ❶ 페이지 접속**:** 개최자가 대회를 개설하고 관리하기 위해 CloudFront를 거쳐 웹 플랫폼에 접속한다.
- ❷ 운영 리소스 제출**:** 대회 진행에 필요한 주제 명세서, 채점 코드, 시각화 리소스 등을 업로드하여 S3에 저장한다.
- ❸ 프론트 활성화**:** 개최자가 등록한 리소스를 바탕으로 해당 대회의 프론트엔드 UI가 구성되고 참가자들에게 노출된다.
- ❹ 대회 활성화**:** Nginx 리버스 프록시가 참가자의 API 요청을 로드밸런싱하여 가용한 Spring Boot 서버로 전달한다.

4.2.2. 소프트웨어 아키텍처

프론트엔드 **(Presentation Layer)**



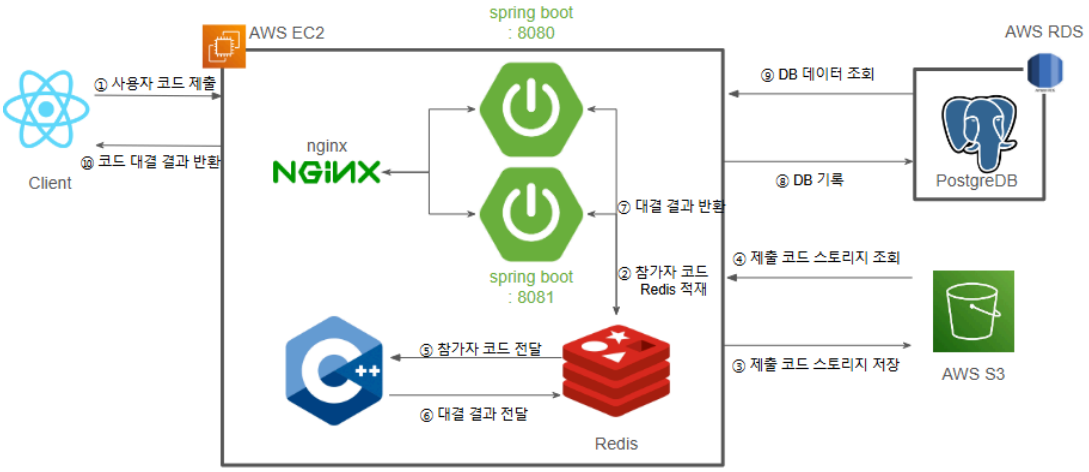
[그림3] 프론트엔드 구조

본 시스템의 프론트엔드와 백엔드 통신 흐름은 다음과 같이 진행된다. 먼저 AWS S3에 호스팅된 정적 리소스가 CloudFront를 거쳐 프론트엔드 환경으로 전달되며 화면을 구성한다.

참가자는 렌더링된 웹 페이지에 접속하여대회 목록 및 상세 정보 확인을 하고, 대회에 참가해 플랫폼이 제공하는 웹 에디터 환경에서 대결에 사용할 배틀 코드를 작성한다. 작성이 완료된 코드는 JSON 형식으로 변환되어 백엔드 서버로 전송된다.

백엔드 서버는 Rest Controller, Service Layer, Data Access Layer로 이어지는 3-Layer 아키텍처를 통해 해당 API 요청의 유효성을 검증하고 비즈니스 로직을 처리한다. 백엔드에서 로직 처리가 완료되면 그 결과값이 RESTful API 형태로 프론트엔드에 반환되며, 화면에서 응답 데이터를 수신하여 참가자에게 대전 결과, 리더보드 등의 요청 정보를 시각적으로 나타낸다.

백엔드 **(Application Layer)**



[그림4] 백엔드 구조

클라이언트의 API 요청이 EC2에 도달하면 Nginx 리버스 프록시가 이를 수신한다. Nginx 뒤에는 Spring Boot 인스턴스 두 대(8080, 8081)가 구성되어 있으며, Blue-Green 배포 방식으로 교대 운영한다. 새 버전 배포 시 비활성 인스턴스에 먼저 배포한 뒤 Nginx의 라우팅을 전환하여, 사용자 요청이 중단되지 않는 무중단 배포를 실현한다.

Spring Boot 서버는 대회 관리, 코드 제출, 사용자 인증(JWT) 등의 비즈니스 로직을 처리하며, 데이터의 성격에 따라 세 가지 저장소에 분산하여 저장한다. 관계형 데이터(사용자, 대회, 대전 결과)는 AWS RDS의 PostgreSQL에 저장하고, 제출된 배틀 코드 파일과 대회 자료는 AWS S3에 업로드한다.

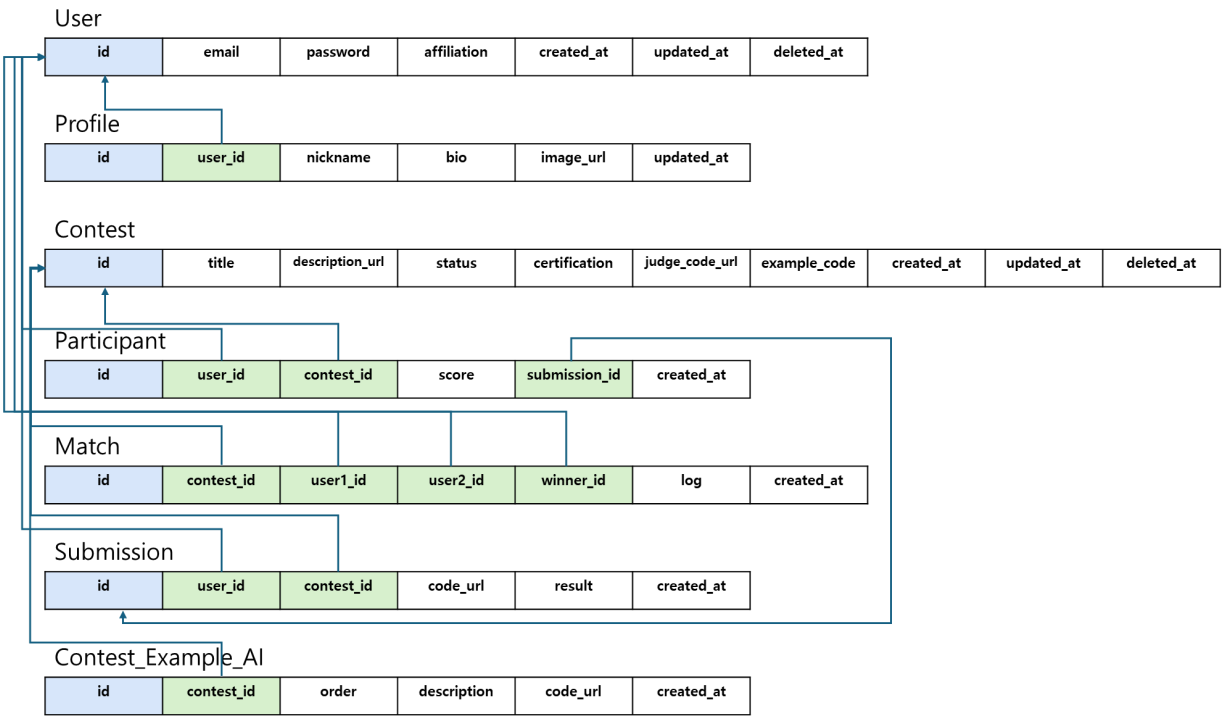
채점 흐름을 보면, 참가자가 코드를 제출하면 Spring Boot가 S3에 코드를 저장한 뒤 Redis 작업 큐에 채점 요청을 등록한다. C++ 채점 서버가 Redis에서 요청을 꺼내 S3에서 코드를 가져와 Docker 컨테이너 내에서 대전 시뮬레이션을 실행하고, 결과를 다시 Redis를 통해 Spring Boot에 반환한다. Docker 컨테이너를 활용하여 참가자 코드를 격리 실행함으로써 보안과 공정성을 보장한다. 이 비동기 구조를 통해 채점 작업이 API 서버를 블로킹하지 않으며, 다수의 채점 요청을 순차적으로 안정적으로 처리할 수 있다.

데이터 계층

- PostgreSQL (AWS RDS): 사용자, 대회, 코드 제출, 대전 결과 등 관계형 데이터를 저장한다. 대전 턴별 로그는 JSONB 타입으로 유연하게 저장한다.

- Redis: Spring Boot와 C++ 채점 서버 간 작업 큐(코드 제출 → 채점 요청 → 결과 반환), 사용자 세션 관리의 세 가지 용도로 활용한다.
- AWS S3: 제출된 배틀 코드의 영구 저장, 대회 명세서, 채점 코드, 시각화 리소스 등 파일을 저장한다.

4.2.3. 데이터 정의



[그림5] DB Schema

해당 그림은 본 프로젝트에서 도출한 데이터베이스 스키마 구조이다.

본 프로젝트의 데이터베이스 아키텍처는 다른 참가자의 코드와 대결하는 형식의 AI 코딩 배틀이라는 핵심 도메인 요구사항을 관계형 데이터베이스 상에 효율적으로 구현하는 데 중점을 두고 설계했다. 먼저, 계정 보안 및 쿼리 성능 최적화를 위해 사용자 계정 엔티티를 인증을 위한 User 테이블과 전시 목적의 Profile 테이블로 분할하였다. 이를 통해 로그인 등 권한 검증 시 발생할 수 있는 불필요한 데이터 로딩을 최소화하였다. 또한, 대회의 상세 설명, 채점 로직 코드, 참가자의 제출 코드 등 대용량 텍스트 및 파일 데이터는 데이터베이스 내부에 직접 저장하는 대신 URL 형태로 관리하여 S3로 저장을 위임했다. 이는 데이터베이스의 I/O 부하를 경감시키고 전체적인 응답 속도를 향상시키기 위함이다.

핵심 기능인 배틀 및 채점 시스템은 확장성과 유연성을 고려하여 테이블을 구성했다. 대회 내에서 두 코드가 배틀 구조는 Match 테이블로 정의하였으며, 승패 결과뿐만 아니라 턴제 배틀의 상세 로그를 함께 기록하여 추후 프론트엔드 측에서 리플레이 시각화를 지원하도록 했다. 아울러 다수의 코드 제출이 발생하는 알고리즘 대회의 특성을 반영하여 Submission 테이블에 모든 제출 및 채점 이력을 보존하고, Participant 테이블을 통해 참가자의 최종 점수 및 랭킹에 반영될 대표 코드를 분리하여 관리하는 구조를 채택했다. 이와 함께 참가자의 원활한 진입을 돕기 위해 난이도별 튜토리얼 및 기준 봇(Baseline) 역할을 수행할 Contest_Example_AI 테이블을 설계하여 시스템의 사용성을 강화하였다.

Column	Type or Constraint	Description
id	int, PK	사용자 식별자
email	string, unique	사용자 이메일
password	string	사용자 비밀번호
affiliation	string	사용자 소속
created_at	datetime	사용자 가입 시각
updated_at	datetime	사용자 정보 변경 시각
deleted_at	datetime	사용자 회원탈퇴 시각

[표 7] 테이블 - 사용자(User)

Column	Type or Constraint	Description
id	int, PK	프로필 식별자
uder_id	int, FK	사용자 FK
nickname	string	사용자 닉네임
bio	string	사용자 상세 설명
image_url	string	사용자 프로필 이미지, url
updated_at	datetime	사용자 프로필 변경 시각

[표 8] 테이블 - 사용자 프로필(Profile)

Column	Type or Constraint	Description
id	int, PK	대회 식별자
title	string	대회 제목
description_url	string	대회 상세 명세, url
status	int	대회 진행 상태
certification	boolean	인증 대회 여부
judge_code_url	string	대회 채점 코드, url
example_code	string	스켈레톤 코드, url
created_at	datetime	대회 생성 시각
updated_at	datetime	대회 수정 시각
deleted_at	datetime	대회 삭제 시각

[표 9] 테이블 - 대회(Contest)

Column	Type or Constraint	Description
id	int, PK	참가자 식별자
user_id	int, FK	사용자 FK
contest_id	int, FK	대회 FK
score	int	해당 대회에서의 레이팅
submission_id	int, FK	최종 제출 코드, FK
created_at	datetime	참가자 대회 참가 시각

[표 10] 테이블 - 참가자(Participant)

Column	Type or Constraint	Description
id	int, PK	매치 식별자
contest_id	int, FK	대회 FK
user1_id	int, FK	사용자1 FK
user2_id	int, FK	사용자2 FK
winner_id	int, FK	승자 FK
log	string	대전 기록 로그
created_at	datetime	참가자 코드 매치 시각

[표 11] 테이블 - 매치(Match)

Column	Type or Constraint	Description
id	int, PK	코드 제출 식별자
user_id	int, FK	사용자 FK
contest_id	int, FK	대회 FK
code_url	string	제출 코드, url
result	string	예시 AI 대전 결과
created_at	datetime	참가자 코드 제출 시각

[표 12] 테이블 - 코드 제출(Submission)

Column	Type or Constraint	Description
id	int, PK	대회 샘플 AI코드 식별자
contest_id	int, FK	대회 FK
order	int	예시 AI코드 순서
description	string	예시 AI코드 설명

code_url	string	예시 AI코드, url
created_at	datetime	대회 샘플 AI코드 제출 시각

[표 13] 테이블 - 대회 샘플 AI(Contest_Example_AI)

4.2.4. API 정의

- 회원(인증)

기능	**Path**	**send data**	**Method**	**Description**
회원가입	/auth/signup	email, password, nickname	POST	서비스 회원가입
회원탈퇴	/auth/signout	password(본인 확인)	PATCH	회원탈퇴
로그인	/auth/login	email, password	POST	로그인(JWT 토큰 발급)
로그아웃	/auth/logout	refreshToken	POST	로그아웃
토큰 갱신	/auth/refresh	refreshToken	POST	토큰 갱신

[표 14] 회원가입 및 인증

- 회원(프로필)

기능	**Path**	**send data**	**Method**	**Description**
내 프로필 조회	/me/profiles		GET	자신의 프로필 조회
내 프로필	/me/profiles	nickname, profileImage	PATCH	자신의 프로필 업데이트

수정				
타 회원 조회	/{memberId}/profiles		GET	타 회원 프로필 조회

[표 15] 회원 프로필

- 대회

기능	**Path**	**send data**	**Method**	**Description**
대회 생성	/contests	title, description, category(CERTIFIED|NON_CERTIFIED), gameRule, startDate, endDate, maxParticipants	POST	대회 규칙, 방식(인증,비인증),일정, 참가자 수 등을 정해 대회 생성
대회 상세 조회	/contests/{contestId}		GET	대회 상세 정보 조회 (규칙, 기간, 참가자 수 등)
대회 정보 수정	/contests/{contestId}	title, description, startDate, endDate 등 수정할 필드 PUT 대회 정보 수정 (ORGANIZER 본인만)	PUT	대회 정보 수정. 이름, 설명, 시작/종료 날짜 등
대회 참가	/contests/{contestId}/join	email	POST	대회 참가 신청

신청				
참가자 목록	/contests/{contestId}/participants	Query: page, size	GET	참가자 목록 조회
웹소켓 연결	/contests/websocket	JWT token	POST	유저와의 중간 대전 결과 공지를 위한 웹소켓 연결 요청

[표 16] 대회 개최 및 참가

- 코드 제출

기능	**Path**	**send data**	**Method**	**Description**
배틀 코드 제출	/contests/{contestId}/submissions	userId, language, code	POST	배틀 코드 제출
내 제출 이력	/contests/{contestId}/submissions	Query: page, size	GET	내 제출 이력 조회

[표 17] 코드 제출

5. 과제 수행 계획

5.1. 업무분장

5.1.1. 개발 업무 분장

팀원	역할	담당 영역
박서윤	Frontend	• React + TypeScript 기반 SPA 개발 • UI/UX 설계 및 구현(대회 목록, 코드 에디터, 리더보드, 대전 시각화) • Zustand 전역 상태 관리

		● CloudFront + S3 프론트엔드 배포 환경 구성
이재혁	Backend / Scoring	● Spring Boot REST API 개발(대회 관리, 코드 제출, 매치 스케줄링) ● C++ 채점 서버 구현, Redis 작업 큐를 통한 Spring Boot-채점 서버 간 비동기 통신 구현 ● Docker 기반 코드 격리 실행 환경 구성 ● Nginx 무중단 배포 환경 구성
안주성	Backend / Infra	● Spring Boot REST API 개발(인증, 리더보드, 결과 관리) ● PostgreSQL 데이터베이스 설계 및 구현 ● JPA/QueryDSL 데이터 접근 계층 구현 ● AWS 인프라 구성(EC2, RDS, S3) ● GitHub Actions CI/CD 파이프라인 구축

[표 18] 개발 업무 분장

5.1.2. 비 개발 분야 업무분장

팀원	담당 영역
박서윤	● UI 에셋 제작 ● 컨셉 및 기획제안 발표 담당 ● 회의록 작성 관리 ● 기획 보고서 작성
이재혁	● 재무 및 회계 담당 ● 깃허브 프로젝트 관리 ● 구글 Docs 문서 관리 ● 기획 보고서 작성
안주성	● 멘토와의 소통 관리 ● 슬랙 채널 관리 ● 기술 발표 담당 ● 기획 보고서 작성

[표 19] 비 개발 업무 분장

5.2. 협업 및 데브옵스 자동화

5.2.1. 커밋 컨벤션

작업 내역을 효율적으로 관리하고 추적하기 위해 깃허브 사용에 팀 내 커밋 규칙을 정하고 따를 계획이다. 아래는 현재 팀 내에서 정한 커밋 규칙이다.

타입 이름	내용
feat	새로운 내용에 대한 커밋
fix	버그 수정에 대한 커밋
build	빌드 관련 파일 수정 / 모듈 설치 또는 삭제에 대한 커밋
chore	그 외 자잘한 수정에 대한 커밋
ci	ci 관련 설정 수정에 대한 커밋
docs	문서 수정에 대한 커밋
style	코드 스타일 혹은 포맷 등에 관한 커밋
refactor	코드 리펙토링에 대한 커밋
test	테스트 코드 수정에 대한 커밋
perf	성능 개선에 대한 커밋

[표 20] 커밋 컨벤션

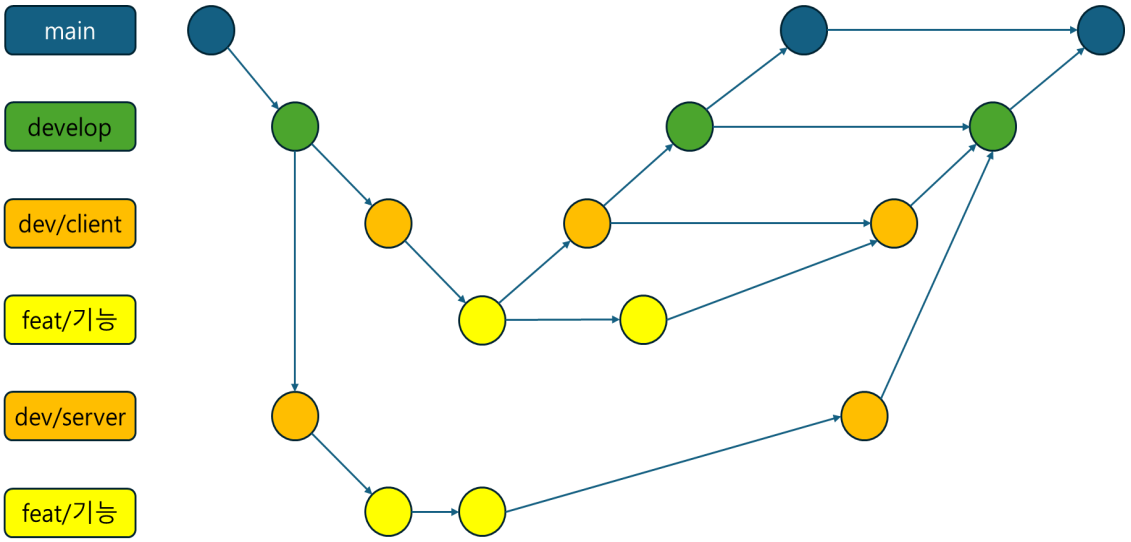
5.2.2. 브랜치 전략

브랜치의 역할을 명확히 정의하여 상위 브랜치로 갈수록 엄격한 품질 관리가 이루어지도록 설계한다.

- main : 언제든 배포 가능한 최종 상태. develop에서 검증 완료된 코드만 머지한다.
- develop (Staging): 스테이징 환경. 모든 기능 구현(feat)이 모여 통합 테스트를 거치는 브랜치이다.
- dev/server & dev/front (Integration): 모노레포 환경에서 파트별 독립적인 개발 및 테스트 환경을 보장하기 위한 중간 집결지. 파트별 의존성이 강한 작업을 우선 통합하고 자체 검증할 때 사용한다.

- feat/~~ (Topic): 개별 기능 단위 개발 브랜치. 목적지에 따라 dev/server나 dev/front로 PR하여 머지한다.

브랜치 간의 대략적인 관계는 아래와 같다.



[그림6] 브랜치 관계도

5.3. 성과 측정 방안

5.3.1. 정량적 KPI

No	분류	KPI 지표명	측정 목적 및 산출 기준	목표
1. 서비스 활성화 및 유저 인게이지먼트 (참가자 중심)				
1	서비스 활성화	MAU (월간 활성 사용자 수)	플랫폼에 실제로 유입되는 사용자 규모를 파악하여 서비스 성장세를 측정한다.	런칭 6개월 내 MAU 1,000명
2	서비스 활성화	대회 당 평균 제출 횟수 (Submission Rate)	참가자가 코드를 지속적으로 수정·발전시키는지 측정한다.	인당 평균 10회 이상
2. 대회 개최 및 B2B 제휴 (개최자/기업 중심)				
4	B2B 제휴	월간 대회 개최 수 (인증/비인증)	수익 모델의 핵심인 대회 개최 빈도를 검증한다.	월평균 인증 3건, 비인증 10건

5	B2B 제휴	B2B 교육기관/기업 제휴 건수	교육기관 서비스 제공료 기반 수익 시장(SOM) 달성을 위한 직접적인 영업 지표이다.	연내 제휴 2곳 체결
6	B2B 제휴	개최자 만족도 및 재개최율	개최자(기업 인사팀, 인디게임 개발자)가 원하는 인재/홍보 효과를 얻었는지 측정한다.	재개최율 30% 이상
3. 시스템 성능 및 운영 안정성 (기술 중심)				
7	시스템 성능	평균 채점 대기 시간 (Queue Time)	C++ 채점 서버 및 Docker 컨테이너의 부하 처리 능력을 측정한다.	문제별로 상이한 실행 제한 시간(Time Limit)을 제외한 코드 제출 후 결과 반환 3초 이내
8	시스템 성능	시스템 가동률 (Uptime)	AWS 인프라(EC2, Nginx Blue-Green, CloudFront)의 무중단 운영 안정성을 검증한다.	99.9% Uptime 유지
9	시스템 성능	테스트 커버리지 (Test Coverage)	JaCoCo를 활용하여 백엔드 핵심 비즈니스 로직(대회, 채점)의 테스트 품질을 관리한다.	핵심 도메인 커버리지 80%

[표 21] 정량적 KPI

5.3.2. 정성적 KPI

No	분류	KPI 지표명	측정 방법 및 기준	목표
1. 사용자 만족도 (참가자 중심)				
1	참가자 만족도	대회 공정성 체감도	대회 종료 후 참가자 설문을 통해 AI 허용 환경의 공정성에 대한 만족도를 측정한다.	5점 만점 중 3.5 이상
2	참가자 만족도	플랫폼 사용성 (UX) 만족도	코드 에디터, 대전 시각화, 리더보드 등 주요 기능의	5점 만점 중 3.5 이상

			사용 편의성을 설문으로 평가한다.	
3	참가자 만족도	대전 시각화 몰입도	대전 리플레이 시각화가 참가자의 코드 개선 동기에 기여하는 정도를 측정한다	긍정 응답률 70% 이상
2. 개최자/기업 만족도(개최자 중심)				
4	개최자 만족도	개최자 만족도 및 재개최 의향	개최자가 대회 운영 과정(자료 업로드, 중간 결과 발표, 최종 결과 관리)에 만족하는지 측정한다.	재개최 의향률 50% 이상

[표 22] 정성적 KPI